



AI for Theological Inquiry

Lecture 2 — From LLMs to agents

Huihuo Zheng
Wheaton College

September 8, 2026

huihuo.zheng@wheaton.edu

Where we are: from the wired-up box to the box itself

✓ Last week

- The three pillars: agent · RAG · eval
- Installed a coding agent (Claude Code / Codex)
- Felt the **plan** → **act** → **observe** loop

You met the box *already wired to tools*.

🔍 This week

- Open the hood on the **raw model**
- What it *is*, and why it hallucinates
- Then add the four things that make an **agent**

Today you meet the box *itself* — on ALCF.

Last week = the box pointed at tools. This week = the box, on its own.

Today: open the box, then add what makes an agent



Part 1 — Concept

- Next-token prediction (recap, deeper)
- Embeddings: meaning as geometry
- Temperature: the same box, wilder
- **Hallucination** — and why
- LLM → agent: the four additions

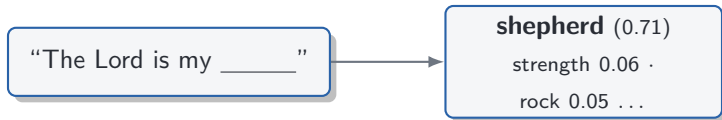


Part 2 — Hands-on

- Log in to the **ALCF inference endpoint**
- Talk to an **open-weight** model directly
- Temperature sweep → watch it diverge
- Provoke a hallucination on purpose

~40 min concept · ~40 min lab · ~20 min project touchpoint.

The one idea: a language model predicts the next token



It outputs a *probability over every possible next token* — then picks one, appends it, and repeats.



What that means

It has no “facts” stored as facts. It has **patterns** of which words follow which — learned from a huge amount of text.



The consequence

It optimizes for **plausible**, not **true**. A fluent wrong answer scores as well as a right one *to the model*.

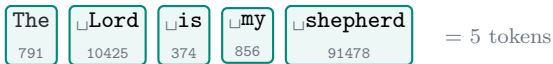
Everything else today — embeddings, temperature, hallucination — follows from this one line.

First it turns words into numbers: tokens & embeddings



Real output of the **Meta-Llama-3.1-8B** tokenizer (128k vocab) — the model you call on ALCF.

A familiar phrase — usually ~one token per word (and the `▯` space is *part of* the token):

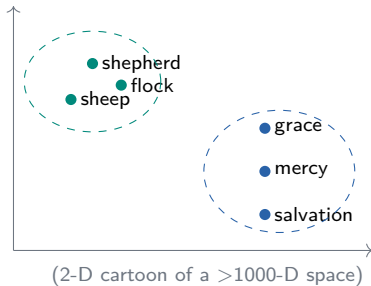


A proper name shatters into *word-pieces* — and even “grace” is not one token:



Each token then becomes an **embedding** (a vector) — the hinge to Week 4, where retrieval is “find the nearest vectors.”

Meaning as geometry: why embeddings matter for us



- Related words land **near** each other — the model learned this from usage, not a dictionary.
- “Distance” becomes something you can **compute**: cosine similarity between two vectors.
- Ask “*which verses are closest to this question?*” and you get real passages back.

That query is exactly what a RAG retriever does —
Week 4.

Embeddings are how a machine “measures” meaning.

Same box, one knob: temperature



The model always produces a probability distribution. **Temperature** controls how boldly you sample from it: near 0 you get the safe, reproducible answer; high, you get variety — and more chances to go off the rails.

For scholarship you usually want *low* temperature — and you *report* the number.

The money moment: confident, fluent, and wrong

 Ask a small model for an obscure or made-up verse — it will invent one, in flawless biblical cadence.

Why it happens

It is completing a **pattern**, not looking something up. “A verse citation” has a shape; it fills the shape convincingly whether or not the verse exists.

What fixes it

Don't ask it to *recall* — give it the **real text** and ask it to *use* that. Grounding = **RAG** (Wk 4).
Checking = **evaluation** (Wk 6).

In theology a fabricated reference is a **scholarly and pastoral failure**. You will reproduce this live today — so you never forget it.

So what turns this predictor into an *agent*?

On its own the model answers once and stops. Add four things and it can **pursue a goal**:

Tools

Call a retriever, a calculator, a code runner, the web — **reach outside itself**.

Memory

Carry state across steps — what it found, what it decided.



Plan–act– observe

Decompose, act, *look at the result*, decide the next step.

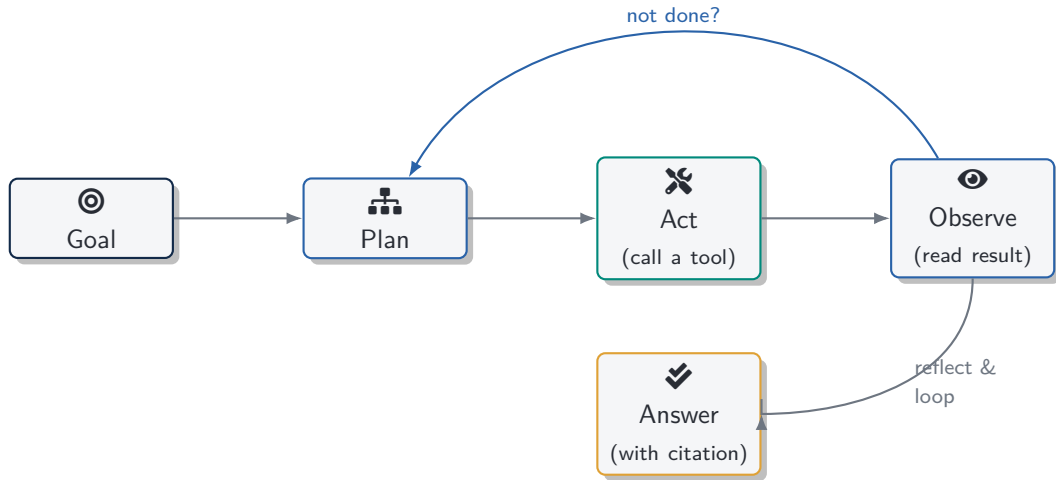


Autonomy

Run many steps toward a goal without a human at each one.

The model you talk to today is the **engine**. An agent is that engine *plus a loop*.

The loop: plan → act → observe → repeat



Today we isolate the very first box — the raw *Act*→*Observe* on one model call.

Why an *open* model, and why on ALCF

Reproducible

“Llama-3.1-8B, temp=0, ALCF” is a **disclosable** fact. “ChatGPT, last spring” is not. Your appendix needs the former.

Governed

A licensed translation or private corpus **stays on ALCF** — it never leaves for a vendor. Matters once we load your texts (Wk 3).

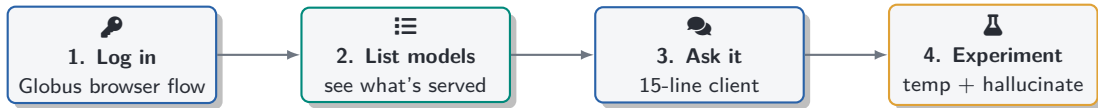
Yours

Free with your ALCF account — no personal API keys on shared machines, and the **same client all term**.

The endpoint is **OpenAI-compatible**: the ~15 lines you write today become the *retriever's brain* in Week 4, unchanged. One model client, one corpus, one agent — all term.

Part 2 — Hands-on: talk to the model directly

i The **ALCF inference endpoint** serves open-weight models (Llama, Qwen, ...) behind a simple web API. No cluster, no queue — just messages in → completion out. This is the raw box from Part 1.



Notebook + one-page handout are in the course materials: `tutorials/week02_alcf_inference/`.

Step 1 — Authenticate (once, good for ~48 hours)

Get the helper & log in

```
# one-time: grab the auth helper
wget https://raw.githubusercontent.com/
argonne-lcf/inference-endpoints/main/
inference_auth_token.py

# opens a browser: log in with ALCF/Globus
python inference_auth_token.py authenticate
```

See what models are served

```
tok=$(python inference_auth_token.py \
    get_access_token)

curl -s https://inference-api.alcf.anl.gov
/\
resource_server/list-endpoints \
-H "Authorization: Bearer $tok"
```

Copy an exact model ID from this list — you'll paste it next.

Token lasts ~48 h; re-run authenticate when calls start returning 401.

Step 2 — The 15 lines that carry the whole term

Python (OpenAI-compatible client)

```
from openai import OpenAI
from inference_auth_token import get_access_token

client = OpenAI(
    api_key=get_access_token(),
    base_url="https://inference-api.alcf.anl.gov/resource_server/sophia/vllm/v1",
)

resp = client.chat.completions.create(
    model="meta-llama/Meta-Llama-3.1-8B-Instruct", # use an ID from list-endpoints
    messages=[{"role": "user", "content": "What does Romans 7:24 say?"}],
    temperature=0,
)
print(resp.choices[0].message.content)
```

`pip install openai` first. Same call shape you'll reuse for retrieval and evaluation later.

Step 3 — Three tiny experiments, three big ideas

Temperature

Same prompt at temp=0 twice (identical) vs temp=1.5 three times (all different). **See determinism vs. sampling.**

Hallucination

Ask for “*Hezekiah 3:16*” (no such book). Watch it confidently produce a verse. **This is the lesson.**

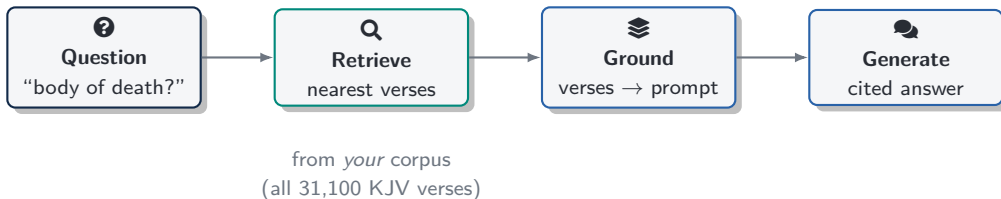
Grounding teaser

Paste the *real* verse into the prompt and ask again — the answer snaps to truth. **That’s RAG, by hand.**

Each is a few lines in the notebook. The arc: the box is **creative** (temperature), **unreliable alone** (hallucination), and **trustworthy when grounded** (teaser) — which is the whole course in miniature.

From teaser to method: this is RAG

You just grounded *one* verse by hand. **RAG** automates that: for any question, first *find* the right passages, then answer *only* from them.



The one rule

The model may use **only** the retrieved text. If the answer isn't there, it must **say so** — not invent.

Same client

Retrieve *and* generate both call the **same** ALCF endpoint from Step 2 — no new tools.

RAG on the whole Bible: with vs. without

WITHOUT RAG — from memory

A small model **paraphrases** Scripture and **invents** chapter:verse. Fluent, familiar-sounding — and often subtly **wrong**.

"Hezekiah 3:16" → a confident, fabricated verse.

WITH RAG — from the text

Retrieves from all **31,100 verses**, quotes **exactly**, and **cites**. Ask for a book that doesn't exist and it **declines**.

"Hezekiah 3:16" → "Not found in the provided verses."

 **The theological stakes:** a paraphrase can drift doctrine; an exact citation can be **checked**. RAG turns the model from a *source* into a *finder* — the scholar stays in charge of the text.

Run it today: `bible_rag/ask_bible.py` — same question, both ways.

How it finds the verse: words now, meaning next

Keyword match (today)

BM25: rank verses by shared words. Pure Python, instant over 31k verses, no setup. Nails phrases (“*still small voice*”) and citations.

Meaning match (next)

Embeddings: rank by *nearest vectors* — Slide 6’s geometry. Catches paraphrase (“*the greatest commandment*”) that shares no keywords.

Same **retrieve** → **ground** → **generate** loop — only the *ranking* step changes. The lab ships with keyword search working out of the box; `build_index.py` adds the embedding upgrade.

Retrieval is the bridge from Slide 6’s “meaning as geometry” to a cited answer.

Project touchpoint (20 min): point the box at *your* question

- Take the **interest statement** you wrote for Week 1. Turn it into **one factual question** an 8B model could plausibly get wrong.
- Run it at temp=0. **Fact-check the answer yourself.** Note exactly where it was fluent-but-wrong.
- That failing question is a **seed for your evaluation set** (Week 6) — keep it.

💡 **The through-line:** today you saw *why* we need grounding and evaluation. Week 3 loads your corpus; Week 4 turns this same client into a retriever over it.

Assignment for next week

1. Run it and save a trace

Get one real completion back from the ALCF endpoint. Save **one screenshot or transcript** of a hallucination you provoked — model ID, temperature, prompt, and the wrong output. (This is your first disclosure-appendix artifact.)

2. Reading

- Anthropic, *Building Effective Agents* (2024) — skim; note the tool-use loop.
- Weng, *LLM-Powered Autonomous Agents* (2023) — read the “planning” and “memory” sections.
- Optional: Wolfram, *What Is ChatGPT Doing...?* — the next-token section.

Next week: the theological corpus — the texts *you* will ground on all term.